Revisão

Fundamentos

O que é programação orientada à objetos?

- Downey define programação orientada a objetos como um estilo de programação em que os dados e as operações que os manipulam são organizados em classes e métodos.
- **Ou seja:** em vez de manter os dados de um lado e, do outro, as funções que trabalham com eles, a POO reúne os dois numa mesma estrutura, a classe, que descreve ao mesmo tempo o que um objeto tem e o que ele faz.

Características que definem a POO:

- Os programas incluem definições de classes e métodos.
- A maior parte da computação é expressa em termos de operações sobre objetos. github
- Os objetos muitas vezes representam coisas do mundo real, e os métodos muitas vezes correspondem às formas como essas coisas interagem.

Vantagens

Código mais conciso e organizado, que reflete a estrutura do problema; relação explícita entre dados e métodos; chamadas mais simples; mais fácil de manter e reutilizar, inclusive por herança.

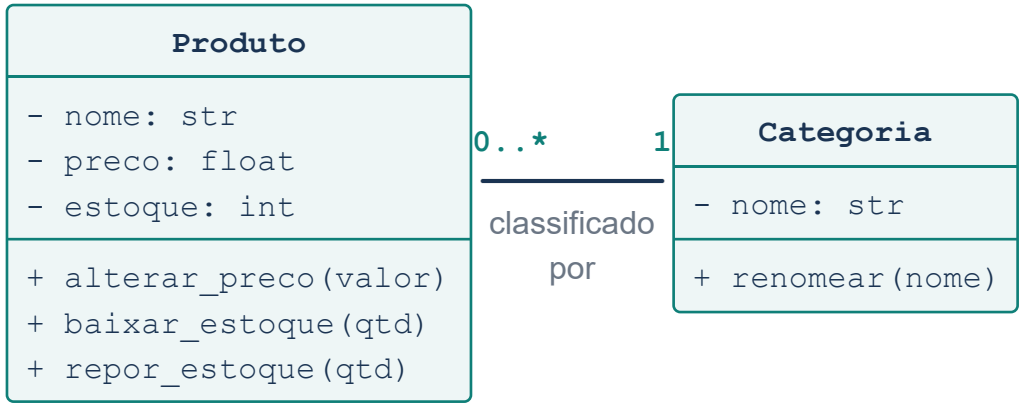
Desvantagens:

A herança pode dificultar a leitura (nem sempre se sabe onde o método foi definido) e a depuração; sobrescrever métodos sem manter a interface pode quebrar o programa; nem sempre é a melhor solução, processamento de dados, programas pequenos e simples podem não necessitar de POO; não é obrigatória.

Diagramas UML

UML (Linguagem de Modelagem Unificada) é a notação-padrão para **representar sistemas orientados a objetos em diagramas**. Para Downey, o diagrama de classes é **uma representação mais abstrata da estrutura do programa**.

Diagrama de classes: Produto e Categoria



Compartimentos: nome · atributos · métodos

Visibilidade: **+** público · **−** privado · **#** protegido

Mostra classes e relações, não objetos individuais.

Relacionamentos

Relacionamento	Notação UML
Associação	Linha contínua
Herança (IS-A)	Seta de triângulo vazio
Agregação	Losango vazio no todo
Composição	Losango cheio no todo
Dependência	Seta tracejada
Multiplicidade	1 · 0..1 · 1..* · *

Outros diagramas: objeto, sequência e caso de uso.

Fontes: DOWNEY, *Pense em Python*, cap. 18 (18.8 – Diagramas de classe) · GUEDES, *UML 2* (Novatec, 2018)

Classes

Segundo Downey, classe é um **tipo definido pelo programador**. O objeto classe funciona como uma **fábrica de objetos**: cada objeto criado a partir dela é uma **instância**.

Pontos-chave

- O Python já tem tipos prontos (**int**, **str**, **list**); a classe cria um tipo novo, como **Produto**.
- Definida com a palavra-chave **class**; nome em CamelCase (convenção da PEP 8).
- **Instanciar** = chamar a classe como uma função: **Produto("Mouse", 100, 20, eletronicos)**.
- **Instância** = objeto que pertence a uma classe: **mouse** e **teclado** têm dados próprios.
- A classe reúne **atributos** (o que o produto tem) e **métodos** (o que ele faz).
- **isinstance(obj, Classe)** verifica se um objeto é instância de uma classe.

exemplo.py

```
class Produto:
    """Um produto à venda na loja."""

    def __init__(self, nome, preco, estoque,
                 categoria):
        self.__nome = nome
        ...

eletronicos = Categoria("Eletrônicos")
mouse = Produto("Mouse", 100, 20, eletronicos)
teclado = Produto("Teclado", 250, 5, eletronicos)
print(type(mouse))    # <class 'Produto'>
print(isinstance(teclado, Produto))  # True
```

Fonte: DOWNEY, *Pense em Python* (Novatec, 2016), cap. 15 – Classes e objetos (15.1 e glossário)

Atributos

Atributo é **um dos valores nomeados associados a um objeto**. Ele é lido pela **notação de ponto**: **objeto.atributo**.

Pontos-chave

- Representam o que o objeto **tem** ou **é**: nome, preço, estoque e categoria.
- **De instância**: criados com **self.x** no **__init__**; cada produto tem os seus valores.
- **Privados** (**__nome**): só a própria classe acessa; de fora, a leitura passa pela property.
- O **estado** muda pelos métodos: **baixar_estoque()** altera **__estoque**.
- Um atributo pode guardar outro objeto: **__categoria** é uma Categoria (associação).
- **De classe**: definidos fora dos métodos e compartilhados por todas as instâncias.

exemplo.py

```
class Produto:
    def __init__(self, nome, preco, estoque,
                 categoria):
        self.__nome = nome           # privado
        self.__preco = 0.0
        self.__categoria = categoria
        ...

   self.__estoque = estoque
mouse = Produto("Mouse", 100, 20, eletronicos)
print(mouse.nome, mouse.estoque)    # Mouse 20
mouse.baixar_estoque(5)     # muda o estado
print(mouse.estoque)        # 15
print(mouse.categoria)      # objeto Categoria
```

Fonte: DOWNEY, *Pense em Python* (Novatec, 2016), cap. 15 (15.2, 15.5) e cap. 18 (18.2 – atributos de classe)

Métodos

Método é **uma função definida dentro de uma classe e chamada a partir das instâncias dela**. Representa o que o objeto **sabe fazer**.

Pontos-chave

- Mesma lógica de uma função, mas definido na classe e chamado pelo objeto: **mouse.alterar_preco(89.90)**.
- O 1º parâmetro é **self**: o objeto que executa a ação (o "sujeito" da chamada).
- **Ação**: muda o estado seguindo uma regra; **alterar_preco()** só aceita preço > 0 (RN1).
- **Consulta**: responde sem alterar nada; **tem_estoque()** devolve True ou False.
- Um método pode chamar outro: **baixar_estoque()** usa **tem_estoque()**.
- **Métodos especiais** (**__init__**, **__str__**) são chamados pelo Python; **__str__** define o **print()**.

exemplo.py

```
class Produto:
    def alterar_preco(self, valor):
        if valor <= 0:
            raise ValueError("Preço inválido")
        self.__preco = float(valor)

    def tem_estoque(self, quantidade):
        return self.__estoque >= quantidade
    def __str__(self):
        return f"{self.__nome} - R$ {self.__preco:.2f}"

mouse.alterar_preco(89.90)    # self = mouse
mouse.tem_estoque(30)         # False
print(mouse)    # Mouse - R$ 89.90 ...
```

Fonte: DOWNEY, *Pense em Python* (Novatec, 2016), cap. 17 – Classes e métodos (17.1–17.6 e glossário)

Objetos

Segundo Downey, objeto é **uma instância de uma classe**: um valor criado a partir do molde, com **dados próprios**. Instanciar é **criar um objeto novo**.

Pontos-chave

- **Classe** é o molde; **objeto** é o que nasce dele: notebook e mouse são objetos de Produto.
- **Estado**: os valores dos atributos naquele momento (estoque = 5).
- **Comportamento**: o que o objeto faz pelos métodos, como baixar_estoque(2).
- **Identidade**: cada objeto é único. O Carrinho usa **is** em __buscar_item() para achar o mesmo produto.
- A **variável** guarda uma referência ao objeto, não uma cópia.
- No **diagrama de objetos**, aparece como notebook : Produto.

src/main.py

```
eletronicos = Categoria("Eletronicos")
notebook = Produto("Notebook", 3500, 5,
                   eletronicos)
mouse = Produto("Mouse", 100, 20, eletronicos)

print(notebook)
# Notebook - R$ 3500.00 (5 em estoque) ...

notebook.baixar_estoque(2)    # muda o estado
print(notebook.estoque)       # 3
print(mouse.estoque)          # 20: outro objeto
```

Fonte: DOWNEY, *Pense em Python* (Novatec, 2016), cap. 15 – Classes e objetos (15.1, 15.2 e glossário)

Construtor

Em Python, o construtor é o método **__init__**: segundo Downey, um método especial **chamado automaticamente quando um objeto é instanciado**.

Pontos-chave

- Serve para **inicializar** o objeto: cria os atributos e define seus valores iniciais.
- Recebe **self** e os dados de criação: nome, preço, estoque e categoria.
- Pode **reaproveitar métodos**: chama **alterar_preco()** para validar o preço (RN1).
- **Valida** antes de terminar: estoque negativo gera **ValueError** (RN2) e o objeto não é criado.
- Não usa **return**: quem devolve o objeto criado é a chamada **Produto(...)**.
- **Destrutor**: **__del__** é chamado quando o objeto vai ser destruído; raramente necessário.

exemplo.py

```
class Produto:
    def __init__(self, nome, preco, estoque,
                 categoria):
        self.__nome = nome
        self.__preco = 0.0
        self.__categoria = categoria
        self.alterar_preco(preco)       # RN1
        if estoque < 0:                 # RN2
            raise ValueError("Estoque negativo")
        self.__estoque = estoque

mouse = Produto("Mouse", 100, 20, eletronicos)
Produto("Fone", -10, 5, eletronicos)
# ValueError: Preço inválido para Fone...
```

Fontes: DOWNEY, *Pense em Python* (Novatec, 2016), cap. 17 (17.5 – O método init) · Python – Modelo de dados (__del__)

PROGRAMAÇÃO ORIENTADA A OBJETOS

Encapsulamento

Encapsular é **reunir dados e operações em um objeto e proteger seus detalhes internos**, expondo só uma interface. Downey: mantenha a **interface separada da implementação**.

Pontos-chave

- **Interface** = o que o programa usa (**preco**, **alterar_preco()**); **implementação** = **__preco**.
- Com uma boa interface, a implementação muda sem quebrar o resto do programa.
- **__preco** é privado: o Python troca o nome (name mangling) e o acesso de fora falha.
- **@property** sem setter: permite ler **mouse.preco**, mas não atribuir um valor.
- A alteração passa por um método com regra: **alterar_preco()** recusa preço ≤ 0 (RN1).
- Não existe privado absoluto: **_nome** é convenção de uso interno; **__nome** dificulta o acesso.

exemplo.py

```
class Produto:
    @property
    def preco(self):                # só leitura
        return self.__preco

    def alterar_preco(self, valor):  # com regra
        if valor <= 0:
            raise ValueError("Preço inválido")
        self.__preco = float(valor)

print(mouse.preco)          # 100.0
mouse.preco = -5            # AttributeError
mouse.alterar_preco(-5)     # ValueError
print(mouse.__preco)        # AttributeError
```

Fontes: DOWNEY, *Pense em Python*, cap. 17 (17.11) e cap. 18 (18.10) · Tutorial Python 9.6 – Variáveis privadas

Associação

Associação é **a ligação entre objetos de classes diferentes**. Downey chama de relação **HAS-A (“tem um”)**: um objeto guarda referências a objetos de outra classe.

Pontos-chave

- **Simples**: os objetos se conhecem e colaboram, mas vivem de forma independente (Produto — Categoria).
- Produto guarda uma **referência** à Categoria: renomear a categoria muda o que o produto mostra.
- **Agregação**: todo-parte em que a parte existe sem o todo (Pedido ◇ ItemCarrinho).
- **Composição**: a parte nasce e morre com o todo (Carrinho ◆ ItemCarrinho).
- **Dependência**: usa outro objeto só como parâmetro, sem guardá-lo como atributo.
- **Multiplicidade**: 0..* produtos para 1 categoria. Não confundir com **herança** (IS-A).

exemplo.py

```
class Categoria:                    # simplificada
    def __init__(self, nome):
        self.__nome = nome
    def renomear(self, nome):
        self.__nome = nome
    def __str__(self):
        return self.__nome
class Produto:
    def __init__(self, ..., categoria):
        self.__categoria = categoria  # tem uma
eletronicos = Categoria("Eletrônicos")
mouse = Produto("Mouse", 100, 20, eletronicos)
eletronicos.renomear("Informática")
print(mouse)    # ... [Informática]
```

Fontes: DOWNEY, *Pense em Python*, cap. 18 (18.8 – HAS-A, IS-A, dependência e multiplicidade) · GUEDES, *UML 2* (Novatec, 2018)

Composição

Composição é uma relação **todo-parte forte**: o todo **cria e guarda** as suas partes, e a parte **não existe sem o todo**. No UML, é o **losango cheio** no lado do todo.

Pontos-chave

- **Todo**: Carrinho. **Parte**: ItemCarrinho.
- Quem **cria** a parte é o todo: adicionar() faz ItemCarrinho(produto, quantidade).
- As partes ficam numa **lista privada**: __itens.
- limpar() esvazia a lista e os itens **deixam de existir**.
- A propriedade itens devolve uma **cópia** da lista: ninguém altera o carrinho por fora.
- Não confundir com **agregação**, em que a parte chega pronta.

src/models/carrinho.py

```
class Carrinho:
    def __init__(self):
        self.__itens = []

    def adicionar(self, produto, quantidade=1):
        ...
        self.__itens.append(
            ItemCarrinho(produto, quantidade))

    def limpar(self):
        self.__itens = []

maria.carrinho.adicionar(notebook, 1)
print(len(maria.carrinho.itens))    # 1
```

Fontes: GUEDES, *UML 2: uma abordagem prática* (Novatec, 2018) · DOWNEY, *Pense em Python*, cap. 18 (18.8)

Herança

Segundo Downey, herança é **a capacidade de definir uma classe nova que é uma versão modificada de uma classe existente**. É a relação **IS-A (“é um”)**: Pix é um Pagamento.

Pontos-chave

- **Classe mãe** (superclasse): Pagamento. **Classes filhas** (subclasses): Pix e Cartao.
- Sintaxe: class Cartao(Pagamento). A mãe vai entre parênteses.
- A filha **herda** atributos e métodos: Pix usa o __init__ e o _valor de Pagamento.
- A filha pode **acrescentar**: Cartao tem __parcelas.
- **super()** chama o método da mãe: super().__init__(valor).
- **Simples**: uma mãe. **Múltipla**: mais de uma, como Pix(Pagamento, Comprovante).

src/models/cartao.py

```
class Pagamento(ABC):
    def __init__(self, valor):
        ...
        self._valor = float(valor)

class Cartao(Pagamento, Comprovante):
    def __init__(self, valor, parcelas=1):
        super().__init__(valor)
        ...
        self.__parcelas = parcelas

cartao_6x = Cartao(3700, 6)
print(cartao_6x.valor)      # 3700.0: herdado
print(cartao_6x.parcelas)   # 6: só do Cartao
```

Fontes: DOWNEY, *Pense em Python*, cap. 18 – Herança (18.7 e 18.8) · Tutorial Python 9.5 – Herança

Classe abstrata

Classe abstrata é uma classe que **não cria objetos**: ela serve de **modelo** para as classes filhas e define **métodos abstratos**, que toda filha é obrigada a escrever.

Pontos-chave

- Em Python: herdar de **ABC** (módulo abc) e marcar o método com **@abstractmethod**.
- Pagamento(100) gera **TypeError**: só Pix e Cartao criam objetos.
- Pode ter **código pronto**: o __init__ valida o valor e guarda _valor.
- **_valor** é protegido (#): a classe e as filhas usam.
- Cada filha escreve o **seu** calcular_total().
- No UML: «abstract», com os métodos abstratos em itálico.

src/models/pagamento.py

```python
from abc import ABC, abstractmethod

class Pagamento(ABC):
    def __init__(self, valor):
        if valor <= 0:
            raise ValueError("...")
        self._valor = float(valor)

    @abstractmethod
    def calcular_total(self):
        ...

# Pagamento(100)
# TypeError: Can't instantiate abstract class
```

Fontes: PYTHON, *abc: classes base abstratas* (docs.python.org) · KEOGH; GIANNINI, *OOP desmistificado* (Alta Books, 2005)

Interface

Interface é um **contrato**: define **quais métodos** uma classe precisa ter, sem atributos e sem código pronto. Quem implementa a interface **tem a capacidade de** fazer aquilo.

Pontos-chave

- O Python não tem a palavra interface: usa-se **ABC só com métodos abstratos**.
- Comprovante define um método: gerar_comprovante().
- Pix e Cartao **implementam** o contrato: cada um escreve o seu comprovante.
- **Classe abstrata**: “é um” (Pix é um Pagamento). **Interface**: “tem a capacidade de”.
- Uma classe pode herdar de uma mãe **e** implementar uma interface: Pix(Pagamento, Comprovante).
- No UML: «interface» e seta tracejada com triângulo vazio.

src/models/comprovante.py e pix.py

```
class Comprovante(ABC):
    @abstractmethod
    def gerar_comprovante(self):
        ...

class Pix(Pagamento, Comprovante):
    def gerar_comprovante(self):
        return (f"Comprovante Pix: R$ "
                f"{self.calcular_total():.2f} "
                f"pagos à vista.")

pix = Pix(3700)
print(pix.gerar_comprovante())
# Comprovante Pix: R$ 3515.00 pagos à vista.
```

Fontes: GUEDES, *UML 2: uma abordagem prática* (Novatec, 2018) · PYTHON, *abc: classes base abstratas* (docs.python.org)

Polimorfismo

Para Downey, é polimórfico o código que **funciona com mais de um tipo** de objeto. A **mesma chamada** serve para objetos de classes diferentes, e **cada um responde do seu jeito**.

Pontos-chave

- **Sobreposição**: Pix e Cartao redefinem calcular_total(), cada um com a sua regra.
- O for chama gerar_comprovante() **sem perguntar o tipo** do objeto.
- Pedido.total() chama calcular_total() sem saber se é Pix ou Cartao.
- Uma forma nova de pagamento entra **sem mudar** o código que usa a lista.
- **Sobrecarga**: o Python não tem; simula-se com parâmetro opcional: Cartao(valor, parcelas=1).
- RN8: Pix com 5% de desconto. RN9: Cartão acima de 3x, 10% a mais.

src/main.py

```
pix = Pix(3700)
cartao_3x = Cartao(3700, 3)
cartao_6x = Cartao(3700, 6)

print(pix.calcular_total())          # 3515.0
print(cartao_3x.calcular_total())    # 3700.0
print(cartao_6x.calcular_total())    # 4070.0

pagamentos = [pix, cartao_3x, cartao_6x]
for pagamento in pagamentos:
    print(pagamento.gerar_comprovante())
# Comprovante Pix: R$ 3515.00 pagos à vista.
# Comprovante Cartão: R$ 3700.00 em 3x ...
```

Fontes: DOWNEY, *Pense em Python*, cap. 17 (17.9 – Polimorfismo) · Tutorial Python 9.5 – Herança

Agregação

Agregação é uma relação **todo-parte fraca**: o todo **recebe partes que já existem**, e a parte **continua existindo sem o todo**. No UML, é o **losango vazio** no lado do todo.

Pontos-chave

- **Todo**: Pedido. **Partes**: os ItemCarrinho que o carrinho já criou.
- O pedido **não cria** os itens: recebe com cliente.carrinho.itens.
- Depois de confirmar(), o carrinho esvazia (RN7), mas o pedido **continua com os itens**.
- confirmar() baixa o estoque de cada item; cancelar() devolve (RN11).
- **Composição**: o todo cria a parte (Carrinho ◆ ItemCarrinho).
- **Agregação**: a parte chega pronta (Pedido ◇ ItemCarrinho).

src/models/pedido.py

```
class Pedido:
    def __init__(self, numero, cliente,
                 forma_pagamento, parcelas=1):
        ...
        self.__itens = cliente.carrinho.itens

pedido = Pedido(1, maria, "pix")
pedido.confirmar()

print(maria.carrinho)        # Carrinho vazio.
print(len(pedido.itens))     # 2: ficam no pedido
```

Fontes: GUEDES, *UML 2: uma abordagem prática* (Novatec, 2018) · DOWNEY, *Pense em Python*, cap. 18 (18.8)